Modernizing Legacy Scientific Codebases

An Agentic Approach to GPU Readiness and Technical Debt

Jorge Luis Gálvez Vallejo
jorge.galvezvallejo@anu.edu.au

NCI
Australian National University
Canberra, Australia / eResearch Australasia 2026



1.7 million lines of Fortran.
Zero unit tests.

1 Who writes scientific software?

Mostly the people least equipped to test it

- A graduate student in their third year, and a postdoc who **inherited** the code from someone who left.
- Very good at the science.
- Nobody taught them to build a test harness — and nobody has the time to learn.

> *What happens when that code needs to move to GPUs, and what can an agentic coding system actually do about it?*
>
> —

1 Why GAMESS?

- Quantum chemistry package, history back to the **1980s**.
- I worked on GAMESS throughput for my entire PhD — and I *wanted* to fix it.
- So: why choose it?

My claim

There is no code in the world as hard to fix as GAMESS. It makes MOM6, WRF and others look like a kindergarten.

1 The problem is the feedback loop

Testing the physics vs. testing the code

Testing the physics

Change something, run the code, compare the total energy to a reference. Matches to ten digits? Assume it is fine.

- Slow
- Only covers the paths your molecule exercises
- When it fails, it says almost nothing about *where*

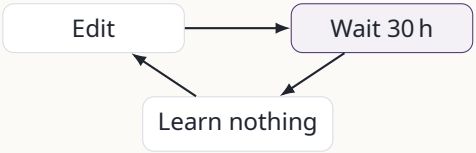
Testing the code

Fast, local, and tells you which line changed behaviour.

- Unit tests, bounds checks, object-code comparison
- Exactly what GAMESS does not have

1 Every change is a thirty-hour bet

- ∼2,000 test inputs; **1,570** run by default.
- A serial run of the **728** that have been timed: **30 hours**.
- So nobody runs it, and people stop refactoring.
- A GPU port is a structural change: **you cannot port a code you are afraid to touch.**



2 What GAMESS looks like

Measured, not estimated

Source	about 1.7 million lines, 440 files
Source form	331 files fixed-form, 108 free (punched-card layout)
COMMON blocks	719 named blocks, 23,646 declarations, copy-pasted into files
Text stored in numbers	12,642 Hollerith constants
Memory	one global array, X, reached by pointer arithmetic
Threading	∼45 blocks made thread-private by a shell script that rewrites the source at build time
Unit tests	**none**

Two matter most for GPUs: the **global memory array** and the **thread-private blocks**.

3 Build the go/no-go before you touch anything

A trustworthy regression gate

- The first thing the agents did was **not change code**. It was **measure**.
- Nine parallel exploration agents, each with one question: the shape of the memory array, where the COMMON blocks go, …
- About **8,000 lines** of reports came back.
- I then organised them by what I knew about GAMESS: deprecated constructs, deleted constructs, COMMON blocks, modules, …

3 The most useful result: things I believed that were wrong

I thought…

The reference outputs were too imprecise for exact comparison.

Actually

They are full precision. They reproduce only **93%** of the time, because they came from another machine.

I thought…

Changing COMMON layouts would break restart files.

Actually

Almost none of it is ever written to disk.

Each would have changed the plan. An agent that read the code carefully found them **in an afternoon**.

3 A reproducibility contract

1 rank, 1 thread

Output is **byte-identical** run to run.

$\Rightarrow$ the **exact gate**.

2 ranks

Not identical: load balancing reorders sums.

$\Rightarrow$ the parallel configuration only gets a **tolerance**.

4 Workflow one: the build system

The best first job for an agent — the answer is checkable: it builds or it does not

- CMake registers every validation deck as its own test, with tiers: tiny, small, medium, slow. **776 → 1,196 tests**.
- The script that rewrote every source file to inject OpenMP directives is **retired**.
- Configuration now **fails** when it names a missing file.

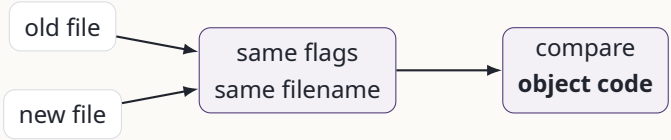
Found by reading all 75 entries

A table of per-file compiler workarounds had **11 entries naming files that do not exist**. Five were typos — those files were silently getting the wrong optimisation flags.

5 Workflow two: transformations you can prove

The idea I most want you to take away

- Fixed-form $\longrightarrow$ free-form, lowercasing, naming every END statement: mechanical. An agent will do them.
- The problem: knowing it did them right across **440 files** when tests cover $\sim$20% of the lines.



Do not use the tests. Compare the object code.

5 Same object code

A stronger claim than a green test suite

Lowercasing pass

441 files
byte-identical object code
zero exceptions

Naming 7,506 END statements

366 files (of 383)
byte-identical object code
zero exceptions

It covers inputs **nobody has written**.

5 And the process found bugs in the tools themselves

- The fixed-to-free converter **silently truncated** statements when a Hollerith string contained an apostrophe.
- The lowercaser turned `8HCCSD(T)` into `8HCCSD(t)`.
- An apostrophe in one string made a quote on a *later* line get lowercased.
- We have 170 payloads with punctuation — not hypothetical.

Division of labour

The agent wrote the tool; the object-code comparison caught the tool; the agent fixed the tool. **The human decided what counted as proof.**

6 Workflow three: re-architecting

The X array — every piece of numerical memory lives in one pool

three lines of Fortran

```
common /fmcom/ x(1)    ! one word
allocate(memory(memlim))
loffs = (loc(memory) - &
         loc(x))/8 + 1
```

- Code indexes a one-element array by the **distance to an unrelated object**.
- Works because nothing checks bounds and the address space is flat.
- Undefined behaviour in every Fortran standard.

6 The cost beyond ugliness

- You can **never turn on bounds checking**.
- Every array-bounds bug in 1.7 million lines is **invisible**.
- You cannot hand that memory to a GPU runtime in any principled way.

6 Why this was tractable

The agent measured before proposing

- **1,230** declarations, **one** shape, **zero** with more than one member.
- The block carries one word that nobody reads: an **address anchor**, not shared storage.
- That made it safe to replace.

Result

x became a real allocatable.
5,123 allocation calls were untouched.

6 What the first bounds-checked run found

Switched on for the first time

143

of 155 tiny tests failed

from eight source lines

- **108** from a single zero-length copy that reads one element past the end of an array.
- Real defects too: a write one past the end of a ten-element array in the input parser, which nothing has ever reported.
- All pre-existing. All confirmed at the baseline. **None findable before.**

7 What this buys you for GPUs

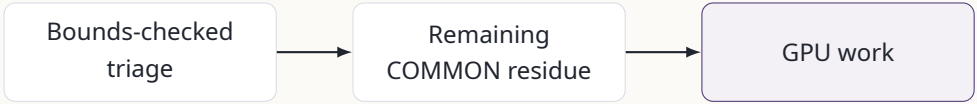
Be precise about the claim

We have not ported GAMESS to a GPU.

We have removed the things that make a port dangerous.

- Global memory array is a real allocatable: **bounds checking works**.
- **714 of 718** COMMON blocks live in modules: data has an owner and a type.
- Math-library matrix multiplies route through a **BLAS wrapper layer** — the seam where a device backend plugs in.
- A small device-management library wraps **CUDA and HIP** behind one interface.
- OpenMP thread-private directives are **in the source**, where a reviewer can see them.

7 What is next



8 What did not work, and what I would tell you to do

1. Go small

Mass mechanical change has failed here before: an earlier sweep of the constants was **reverted five commits later**. Small slices, each with its own gate.

2. Distrust checkers

The agent's own ad-hoc source-level checker raised false alarms twice. **Object-code comparison was the authority**, the checker was not.

3. Some calls are human

Four COMMON blocks declare the same array with **different sizes** in different files. The tool refuses to pick one — correctly. Someone who knows the physics must.

9 Take-away

- The graduate student is not going to learn software engineering in a weekend.
- But an agent can **build the gate**, do the **tedious part**, **prove** it did, and **teach the grad student HOW** to do it without it.
- **Pick the proof first. Then let the agent loose.**

Thank you

Questions?



Jorge Luis Gálvez Vallejo

jorge.galvezvallejo@anu.edu.au